Note 19: Linked List

Instructions

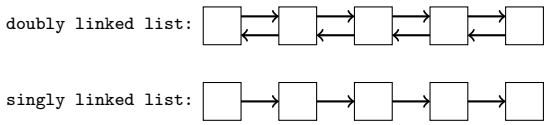
Our objective is to study and define the *linked list* and to construct a container class for its node.

Background

A *linked list* is an alternative data storage structure that, in certain scenarios, provides insertion and removal operations with a constant worst-case runtime of $O(1)$. Unlike arrays, which store elements in contiguous memory and provide random access, linked lists consist of independent memory nodes connected by pointers.

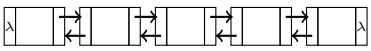
Because a linked list allocates memory dynamically, it uses only the space it needs and does not reserve extra memory for future elements. However, elements have no direct indices, so traversing a linked list requires sequential access starting from the head.

A *node* is the fundamental building block of a linked list. Each node consists of *data*, which holds the actual value, and at least one *link*, which is a pointer to another node. Sequential linked lists are commonly constructed as *singly linked* or *doubly linked* lists.

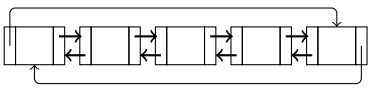


A singly linked list consists of nodes with a single link that points to the next node, so traversal is one-directional. A doubly linked list has two links per node: one points to the next node and the other to the previous node, so traversal is possible in both directions. A doubly linked list can be reduced to a singly linked list by using only one of its links.

A linked list can be either *null-terminated* or *circular*. In a null-terminated doubly linked list, the previous link of the first node and the next link of the last node are both null.



In a circular doubly linked list, the previous link of the first node points to the last node and the next link of the last node points to the first node, forming a loop.



A *sentinel*, a permanent dummy node that is linked to the first and last nodes, can be used to simplify the boundary conditions of circular doubly linked list operations.

Traversal Traversing a linked list uses pointers instead of indices: each step assigns one of the current node's links to itself. To ensure the list is always reachable, traverse with a temporary node pointer rather than the pointer that identifies the list, except when the list is being destroyed.

Deep Copy A deep copy creates an entirely new linked list with duplicated data, so that the new list does not reference the memory of the original and avoids irregular behavior. After the first node of the copy is created, the original and copied lists are each assigned to a temporary node pointer used for traversing. The copy process then traverses the original list and appends a copy of each node to the end of the copied list. At the end of the process, it returns the head of the copied list.

Deletion To delete a linked list, each node must be removed and deallocated one at a time during a forward traversal to prevent memory leaks. A doubly linked list does not need to handle its previous links separately, since every node is deallocated during the traversal.

Coding

Create a header file that includes only the libraries `iostream`, `string`, `sstream`, and `stdexcept`, and defines the generic *Node* class with the following private fields:

- A generic object named *content*.
- A generic *Node* pointer array named *links* of size 2.

and the following public methods:

- `Node()`: assigns the default value of the generic type to *content* and `nullptr` to each element of *links*.
- `Node(const T& obj)`: assigns *obj* to *content* and `nullptr` to each element of *links*.
- `Node(const Node<T>& obj)`: assigns the *content* of *obj* to *content* and `nullptr` to each element of *links*.
- `Node<T>& operator=(const Node<T>& rhs)`: assigns the *content* of *rhs* to *content* and `nullptr` to each element of *links*, and returns `*this`.
- `~Node()`: does nothing.
- `const T& data() const`: returns *content*.
- `T& data()`: returns *content*.
- `const Node<T>* prev() const`: returns *links*`[0]`.
- `Node<T>*& prev()`: returns *links*`[0]`.
- `const Node<T>* next() const`: returns *links*`[1]`.
- `Node<T>*& next()`: returns *links*`[1]`.

Then define the generic functions:

- `template <typename T>`
  `Node<T>* clone(const Node<T>* ptr)`: returns a deep copy of the linked list pointed to by *ptr*.
- `template <typename T>`
  `void clear(Node<T>*& ptr)`: destroys the linked list pointed to by *ptr*.
- `template <typename T>`
  `string toStr(const Node<T>* ptr)`: returns a string listing the elements of the linked list pointed to by *ptr* in the forward direction.

Afterward, create a `.cpp` file that tests the classes as follows:

1. Create an integer *Node* pointer object.
2. Build a linked list with the values 1 through 20, with the object pointing to its head.
3. Display the list by invoking `toStr()` with the object as its argument.
4. Initialize another integer *Node* pointer object to a copy of the original by invoking `clone()`.
5. Verify that both lists have identical values in the same order by traversing them simultaneously and comparing their nodes.
6. If they are the same, display the second list; otherwise, display the message `"Copy unsuccessful"`.
7. Destroy both linked lists by invoking `clear()`.